TFG · SONDA PASIVA DE DETECCIÓN PQC

Analizadores TLS existentes

Búsqueda de implementaciones previas y delimitación de la aportación propia

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **13/10/2026** | Tutoría del miércoles 14/10/2026

1. Conclusión principal

> RESULTADO DE LA BÚSQUEDA
>
> **No existe ningún analizador TLS postcuántico escrito en C sobre libpcap.** Lo más cercano, `tlscensus`, está escrito en Go y no usa libpcap. Las implementaciones en C que sí hay (nDPI, PcapPlusPlus) analizan TLS pero no emiten ningún veredicto sobre seguridad postcuántica.

2. El competidor conceptual: tlscensus

`github.com/bear33/tlscensus · Go · Apache-2.0 · v0.1.0`

Se describe como *passive TLS cryptography inventory for Linux, macOS and Windows*. Observa handshakes de forma pasiva y extrae versiones, cipher suites, grupos de intercambio de claves, algoritmos de firma, ALPN, SNI y huellas JA4, emitiendo **un veredicto de preparación postcuántica por handshake**.

Ha llegado a las mismas conclusiones de diseño

Lo relevante no es que exista, sino que de forma independiente ha adoptado exactamente las decisiones que se dedujeron en este TFG razonando sobre la norma y midiendo 20 dominios:

Decisión de diseño de este TFG	Lo que hace tlscensus
Separar `key_share` de `supported_groups`	*"Reports these separately, so a post-quantum group that is advertised but not used is not counted as post-quantum"*
«Indeterminado» como estado de primera clase	Escalera de cuatro estados: `post_quantum`, **`offered_not_selected`**, **`advertised_only`**, `classical`
El reensamblado TCP es obligatorio	*"A parser that reads one packet drops precisely the handshakes a post-quantum inventory exists to count"*
Los certificados no son visibles en TLS 1.3	Declarado como limitación conocida

Su `offered_not_selected` es el estado INDETERMINADO de este trabajo. Su `advertised_only` es el caso en que el cliente anuncia un grupo postcuántico pero no envía su clave.

Cómo interpretarlo

Como validación externa: las decisiones de diseño de este TFG no son especulativas. Un proyecto independiente, enfrentado al mismo problema, ha tomado las mismas. Es una referencia que conviene citar en la memoria.

Como competencia: existe una herramienta que hace conceptualmente lo mismo. Hay tres diferencias reales:

Diferencia	Detalle
Lenguaje y captura	Go, sin libpcap: en Linux captura con AF_PACKET y en macOS lee `/dev/bpf*`. El enunciado de este TFG pide C con libpcap, y la comparación de rendimiento C frente a Go es un resultado medible para el capítulo de evaluación.
No implementa la patente	No construye el inventario de la reivindicación 10, no mantiene estado persistente por servidor entre conexiones ni distingue las dos categorías de inseguridad.
Unidad de análisis	Clasifica **conexiones**; la patente clasifica **equipos**. La diferencia es precisamente el estado agregado por servidor.

> **RECOMENDACIÓN**
>
> Plantearlo en la tutoría antes de que lo plantee el tutor. Llegar con «he encontrado un proyecto que hace algo parecido y estas son las diferencias» es mucho mejor que que aparezca más adelante.

3. Referencias de implementación en C y C++

PcapPlusPlus — la más útil

`pcapplusplus.github.io · C++ sobre libpcap`

Tiene las dos piezas que faltan por escribir en este TFG:

- **Parseo SSL/TLS** (solo lectura): ClientHello y ServerHello, con huellas JA3 y JA3S. También decodifica certificados X.509.
- **Motor de reensamblado TCP**, con una aplicación de ejemplo `TcpReassembly`. Añade desfragmentación IPv4 e IPv6.

Ese motor de reensamblado es exactamente lo que corresponde implementar en el bloque 3 de este trabajo. Aunque se escriba en C y a mano, conviene estudiar antes cómo lo resuelve una biblioteca madura.

nDPI — la referencia pesada

`github.com/ntop/nDPI · C`

Inspección profunda de paquetes de ntop, con dissector de TLS y huellas JA3/JA4. Es un proyecto enorme, pero su dissector de TLS es la implementación en C más madura disponible.

Esqueletos de sniffer en C

Proyecto	Observación
vichargrave.github.io «Develop a Packet Sniffer with Libpcap»	Tutorial clásico, bien explicado
github.com/baoqger/cPacketSniffer	Captura en C con libpcap
github.com/Vikas2171/Packet-Sniffer	Ethernet, ARP, IPv4, TCP, UDP, ICMP

Ninguno analiza TLS. La sonda de este TFG ya está al nivel de estos esqueletos tras el bloque 1.

4. Lo que no sirve

`libja3` (github.com/jgru/libja3) aparece en las búsquedas como implementación de JA3 en C y parece prometedor. En realidad tiene **un commit, dos estrellas, el README marcado como WIP y la sección de uso vacía con un TODO**. Está abandonado.

> **CRITERIO DE BÚSQUEDA**
>
> Antes de adoptar cualquier proyecto de GitHub como referencia, mirar el historial de commits y la fecha del último. Hay mucho repositorio que parece lo que se necesita hasta que se comprueba su actividad.

5. Referencias académicas y normativas localizadas

Referencia	Utilidad
The Colossus with Feet of Clay: Debunking Encrypted Traffic Classifiers under PQC Evolution `arxiv.org/pdf/2608.22683`	Comportamiento de los clasificadores de tráfico cifrado ante la llegada del postcuántico. Encaja en el estado del arte.
draft-westerbaan-tls-keyshare-recommendations-02 `IETF, abril de 2026`	Propone marcar X25519MLKEM768 como *Recommended* en el registro IANA. Su sección de seguridad articula el argumento del *harvest now, decrypt later* y el del ataque activo con ordenador cuántico. Buena cita para la introducción.

6. Resumen para la tutoría

Punto	Detalle
No hay competencia directa en C	Ningún analizador TLS postcuántico en C sobre libpcap
Existe tlscensus, en Go	Hace conceptualmente lo mismo; se cita como validación y se compara en rendimiento
Coincide en las tres decisiones clave	key_share vs supported_groups, estado indeterminado, reensamblado obligatorio
Diferencias que justifican el TFG	C con libpcap, inventario de la patente, clasificación de equipos y no de conexiones
Referencia de implementación elegida	PcapPlusPlus por su motor de reensamblado TCP; nDPI por su dissector de TLS